



Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Methods for Detecting Attack

Final Project

Submitted by

Natalie Mirelashvili
Faculty of Computer and Information Science
Ben-Gurion University of the Negev
E-mail: miralshv@post.bgu.ac.il

[complete others]

June 2, 2026

1 Related Work

1.1 Prompt injection and automated attack generation

Prompt injection attacks have become a major security concern for LLM-integrated systems because trusted instructions and untrusted content are often processed within the same model context [1, 2]. Greshake et al. [1] showed that attackers can embed malicious instructions in external content, such as web pages or documents, which are later executed when consumed by an LLM application. Subsequent work formalized prompt injection attacks and introduced systematic frameworks for evaluating attacks and defenses across models and tasks [2]. Beyond explicit injections, PromptRobust demonstrated that even small perturbations at the character, word, sentence, or semantic level can significantly affect model behavior [3]. Together, these findings highlight that attack success depends on multiple factors, including the model, task, prompt structure, and location of attacker-controlled input.

Research subsequently shifted from hand-crafted prompt injections toward automated attack generation, motivated by the brittleness and limited transferability of manually designed prompts [4, 5]. Universal adversarial suffixes demonstrated that optimization-based search can automatically generate attacks that transfer across aligned models, including black-box systems [4]. This idea was later extended to LLM-as-a-Judge settings, where JudgeDeceiver searches for adversarial token sequences that cause the judge to prefer a target response [5]. These studies show that effective attacks can be generated automatically rather than manually crafted, making malicious behavior increasingly difficult to identify. As a result, attacks may appear as benign retrieved content, subtle prompt perturbations, or optimized transferable suffixes rather than explicit suspicious instructions [1, 3, 4].

1.2 Defenses for LLM-integrated systems

Existing defenses for LLM-integrated systems are commonly divided into soft and hard defenses [2, 6]. This taxonomy captures the distinction between mitigation techniques applied at inference time and defenses that modify the underlying system design.

Soft defenses. Soft defenses aim to mitigate prompt injection without modifying the underlying model or application architecture [2]. Liu et al. [2] benchmark a diverse set of soft defenses, ranging from prompt modifications to inference-time detection techniques. For indirect prompt injection, BIPIA introduces boundary-awareness mechanisms and explicit reminders that encourage the model to treat retrieved content as data rather than executable instructions [7]. These methods are practical and easy to deploy, but their effectiveness often decreases against adaptive attacks or when models fail to consistently

follow prompt-level safeguards [2, 7].

Hard defenses. In contrast, hard defenses seek to address prompt injection at the architectural level by enforcing stronger trust boundaries, context separation, or input authenticity guarantees [8, 9]. The StruQ framework separates system instructions and user data into structured channels and fine-tunes the model to follow only the instruction channel [8]. By construction, this approach treats prompt injection as a context-separation problem rather than solely a malicious-text detection problem. Shield adopts a broader system perspective by defining integrity, source identification, attack detectability and utility preservation as required safety properties for LLM-integrated applications [9]. Survey studies place these mechanisms within a wider defense toolbox that also includes corpus cleaning, robustness optimization and safe instruction tuning [6].

Overall, recent work suggests a shift from prompt-level mitigation toward architectural defenses that provide stronger guarantees through context separation, authenticated data flows and safer-by-design interfaces [7, 8, 9].

1.3 Threat modeling and automation for LLM systems

LLM-based threat modeling systems are emerging as tools that adapt classical security methods to AI-integrated applications [10]. Threat modeling for LLM-powered applications combines STRIDE, Shostack’s four-question framework and DREAD to reason about risks such as prompt injection, data poisoning, jailbreaking, and information disclosure. Automated LLM integration frameworks extend this idea by using NLP, machine learning, behavioral profiling, and anomaly detection to identify and assess LLM-specific threats[11]. STRIDE GPT illustrates the practical direction of this domain by using LLMs to generate threat models, attack trees, mitigations, and security test cases from system descriptions or code context [12].

This automation builds on earlier work that turns threat models into executable or queryable security artifacts [13]. Formal threat models can be transformed into attack paths and runnable security tests, making the model itself a basis for systematic validation. Security design flaws can also be detected automatically by representing designs as annotated Data Flow Diagrams and applying model query patterns derived from inspection guidelines [14]. The Meta Attack Language supports reusable formal attack descriptions by defining domain-specific languages for threat modeling and attack simulation [15].

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications

with indirect prompt injection,” 2023.

- [2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, USENIX Association, 2024.
- [3] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, “Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts,” 2023.
- [4] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023.
- [5] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, “Optimization-based prompt injection attack to llm-as-a-judge,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.
- [6] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (llm) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.
- [7] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD ’25, Association for Computing Machinery, 2025.
- [8] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “Struq: Defending against prompt injection with structured queries,” 2024. To appear at USENIX Security Symposium 2025.
- [9] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, “Identifying and mitigating vulnerabilities in llm-integrated applications,” 2023.
- [10] S. B. Tete, “Threat modelling and risk analysis for large language model (llm)-powered applications,” 2024.
- [11] B. Gadade and G. A. Patil, “A threat modeling framework for llm system integration leveraging nlp and machine learning,” *Communications on Applied Nonlinear Analysis*, vol. 32, no. 10s, 2025.
- [12] M. Adams and P. Raghavan, “Se radio 648: Matthew adams on ai threat modeling and stride gpt.” Software Engineering Radio podcast, 2024.

- [13] D. Xu, M. Tu, M. Sanford, L. Thomas, D. Woodraska, and W. Xu, “Automated security test generation with formal threat models,” *IEEE Transactions on Dependable and Secure Computing*, vol. 9, no. 4, pp. 526–540, 2012.
- [14] K. Tuma, L. Sion, R. Scandariato, and K. Yskout, “Automating the early detection of security design flaws,” in *Proceedings of the ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems*, MODELS ’20, pp. 332–342, Association for Computing Machinery, 2020.
- [15] P. Johnson, R. Lagerström, and M. Ekstedt, “A meta language for threat modeling and attack simulations,” in *Proceedings of the 13th International Conference on Availability, Reliability and Security*, ARES 2018, pp. 38:1–38:8, Association for Computing Machinery, 2018.